```
  final case class UninitializedState(status: Status) extends State
  final case class OpenState(
      status: Status,
      marketConfirmed: Option[Boolean] = None,     ❶
      fundsConfirmed: Option[Boolean] = None)      ❷
      extends State
  final case class SettledState(status: Status) extends State
  final case class CancelledState(status: Status) extends State
  final case class FailedState(status: Status, reason: String) extends State
  final case class ClosedState(status: Status) extends State
  ...
}
```

❶ Registers if the market is available

❷ Registers if funds were reserved

It is worth noting that `marketConfirmed` and `fundsConfirmed` are relevant only for `OpenState`. No other state keeps track of them; whether the bet is closed, canceled, or failed is recorded in other ways. `Fail` keeps it explicitly in the `reason` and `Close` and `Cancelled` keep it implicitly.

INITIALIZATION

Now let's look at the constructor, mainly to see how it initializes sharding and the timer and passes them along with the context to the command handler:

```
object Bet {
  ...

  def apply(betId: String): Behavior[Command] = {
    Behaviors.withTimers { timer =>                    ❶
      Behaviors.setup { context =>                     ❷
        val sharding = ClusterSharding(context.system)
        EventSourcedBehavior[Command, Event, State](
          PersistenceId(TypeKey.name, betId),
          UninitializedState(Status.empty(betId)),
          commandHandler = (state, command) =>
```